

Importing libraries

```
In [ ]: # Usual Libraries
import nbconvert
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import sklearn

import librosa
import librosa.display
import IPython.display as ipd
import warnings
warnings.filterwarnings('ignore')
import os
```

```
In [3]: import os
general_path = '.'
print(list(os.listdir(f'{general_path}/genres_original/')))
```

```
['reggae', 'classical', 'pop', 'metal', 'rock', 'disco', 'country', 'blues', 'hipho
p', 'jazz', '.ipynb_checkpoints']
```

Exploring audio data

```
In [4]: # Importing 1 file
y, sr = librosa.load(f'{general_path}/genres_original/reggae/reggae.00036.wav')

print('y:', y, '\n')
print('y shape:', np.shape(y), '\n')
print('Sample Rate (KHz):', sr, '\n')

# Verify length of the audio
print('Check Len of Audio:', 661794/22050)
```

```
y: [0.02072144 0.04492188 0.05422974 ... 0.06912231 0.08303833 0.08572388]
```

```
y shape: (661794,)
```

```
Sample Rate (KHz): 22050
```

```
Check Len of Audio: 30.013333333333332
```

```
In [5]: # Trim leading and trailing silence from an audio signal (silence before and after
audio_file, _ = librosa.effects.trim(y)

# the result is an numpy ndarray
```

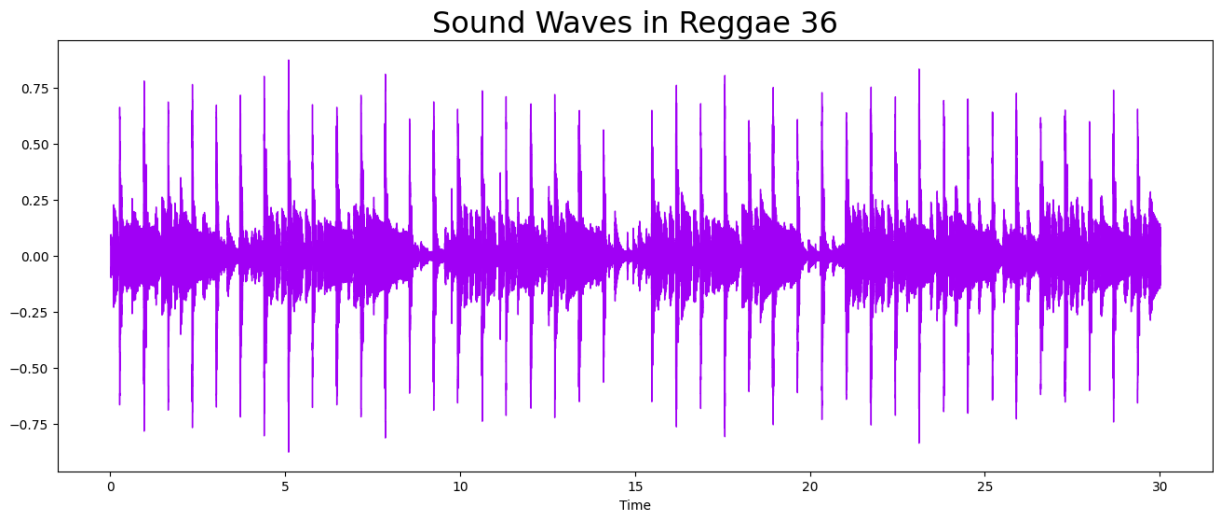
```
print('Audio File:', audio_file, '\n')
print('Audio File shape:', np.shape(audio_file))
```

Audio File: [0.02072144 0.04492188 0.05422974 ... 0.06912231 0.08303833 0.08572388]

Audio File shape: (661794,)

- 2D Representation: Sound Waves

```
In [7]: plt.figure(figsize = (16, 6))
librosa.display.waveshow(y = audio_file, sr = sr, color = "#A300F9");
plt.title("Sound Waves in Reggae 36", fontsize = 23);
```



- Fourier transform

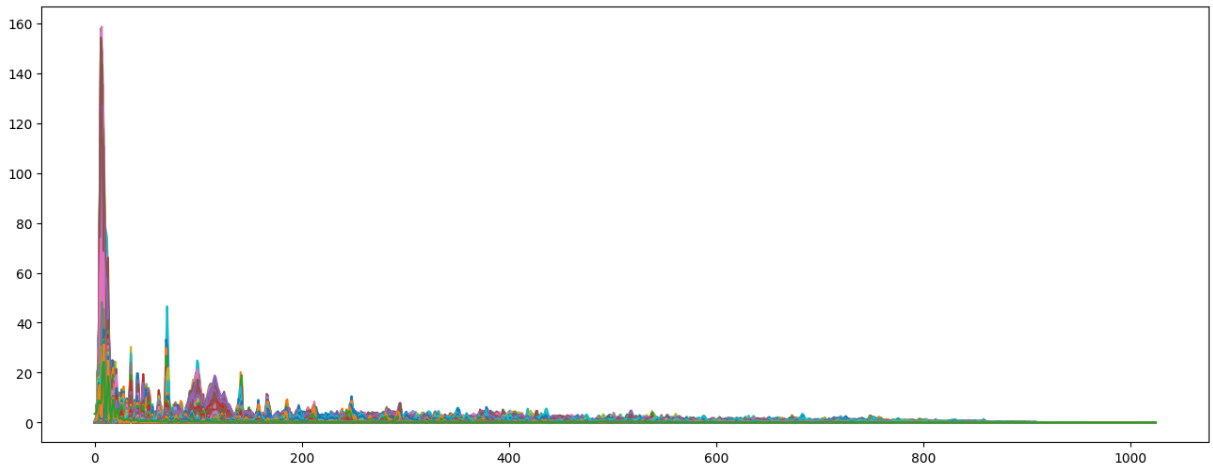
```
In [8]: # Default FFT window size
n_fft = 2048 # FFT window size
hop_length = 512 # number audio of frames between STFT columns (looks like a good d

# Short-time Fourier transform (STFT)
D = np.abs(librosa.stft(audio_file, n_fft = n_fft, hop_length = hop_length))

print('Shape of D object:', np.shape(D))
```

Shape of D object: (1025, 1293)

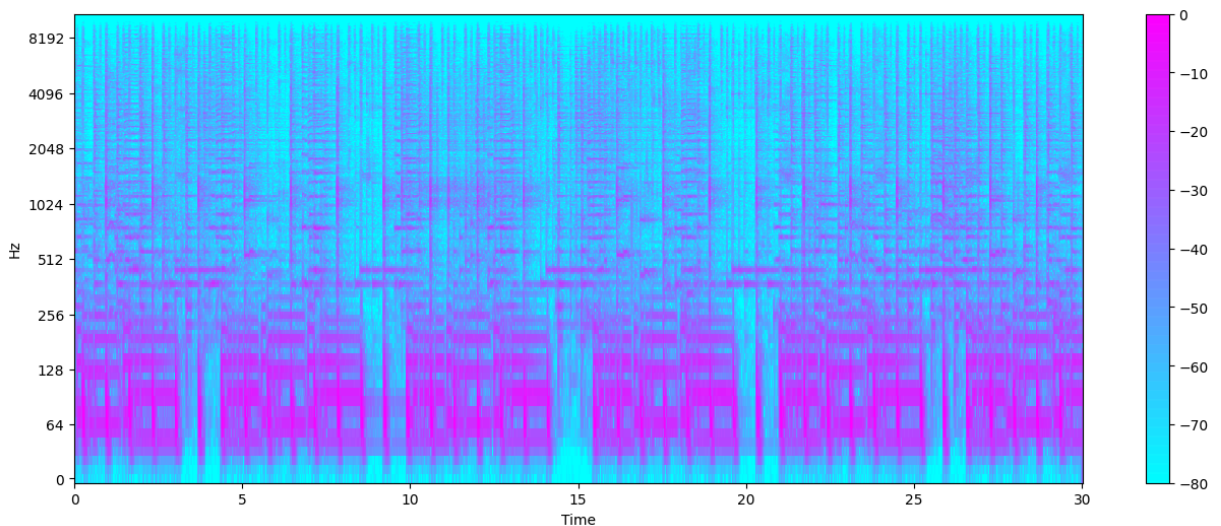
```
In [9]: plt.figure(figsize = (16, 6))
plt.plot(D);
```



- Spectrogram

```
In [10]: # Convert an amplitude spectrogram to Decibels-scaled spectrogram.
DB = librosa.amplitude_to_db(D, ref = np.max)

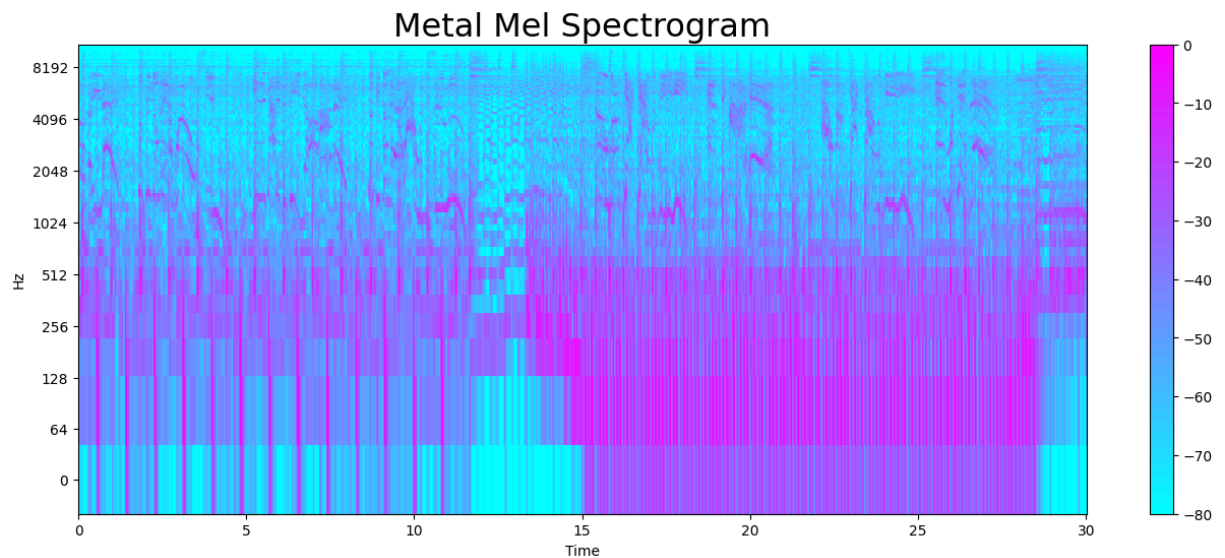
# Creating the Spectrogram
plt.figure(figsize = (16, 6))
librosa.display.specshow(DB, sr = sr, hop_length = hop_length, x_axis = 'time', y_a
                        cmap = 'cool')
plt.colorbar();
```



```
In [12]: y, sr = librosa.load(f'{general_path}/genres_original/metal/metal.00036.wav')
y, _ = librosa.effects.trim(y)

S = librosa.feature.melspectrogram(y=y, sr=sr)
S_DB = librosa.amplitude_to_db(S, ref=np.max)
plt.figure(figsize = (16, 6))
librosa.display.specshow(S_DB, sr=sr, hop_length=hop_length, x_axis = 'time', y_axi
                        cmap = 'cool');

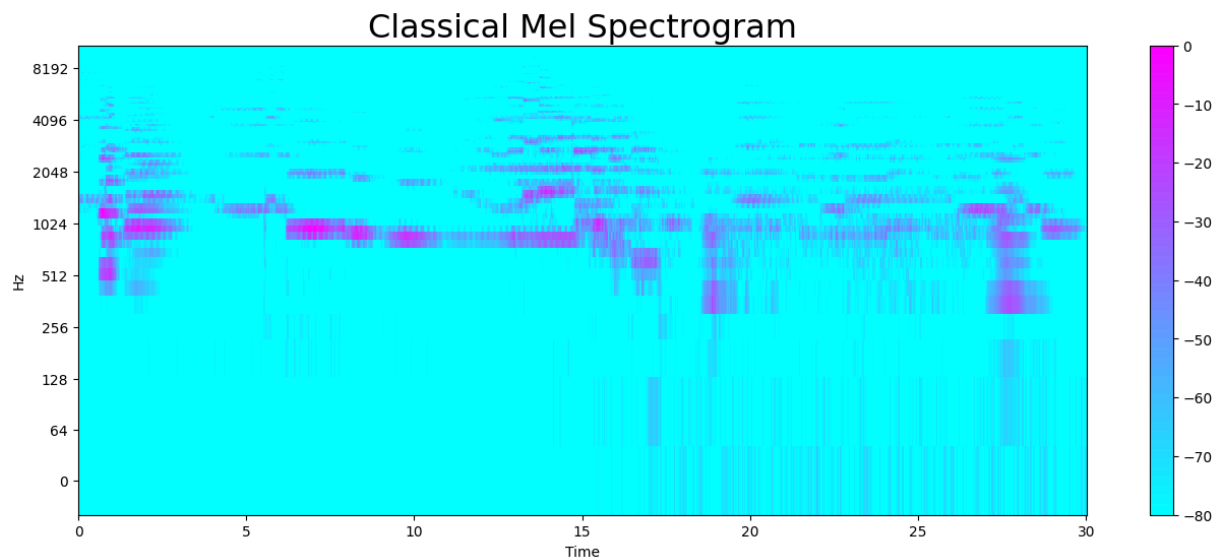
plt.colorbar();
plt.title("Metal Mel Spectrogram", fontsize = 23);
```



```
In [13]: y, sr = librosa.load(f'{general_path}/genres_original/classical/classical.00036.wav')
y, _ = librosa.effects.trim(y)

S = librosa.feature.melspectrogram(y=y, sr=sr)
S_DB = librosa.amplitude_to_db(S, ref=np.max)
plt.figure(figsize = (16, 6))
librosa.display.specshow(S_DB, sr=sr, hop_length=hop_length, x_axis = 'time', y_axis = 'mel',
                        cmap = 'cool');

plt.colorbar();
plt.title("Classical Mel Spectrogram", fontsize = 23);
```



Audio features

- Zero crossing rate

```
In [14]: # Total zero_crossings in our 1 song
zero_crossings = librosa.zero_crossings(audio_file, pad=False)
```

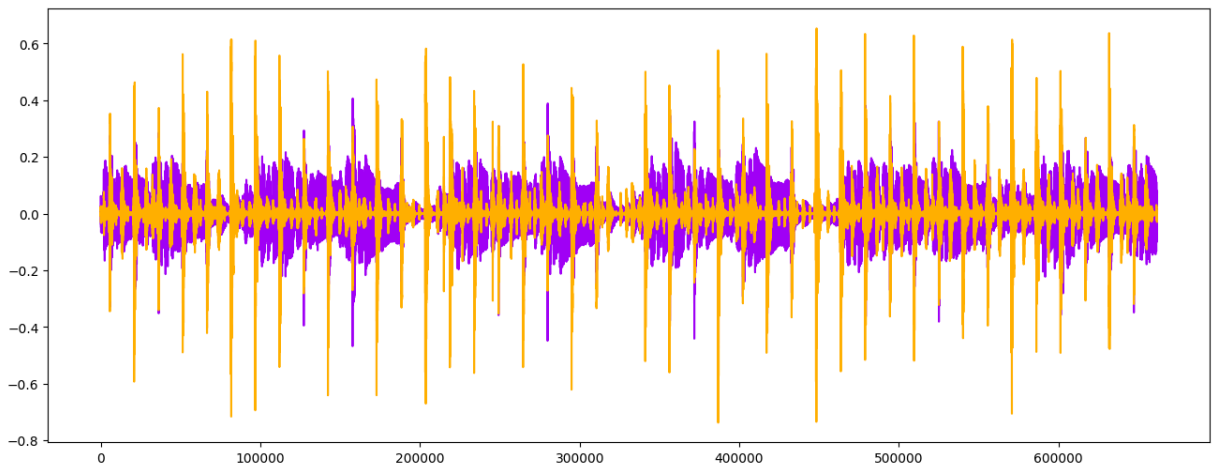
```
print(sum(zero_crossings))
```

39232

- Harmonics and Perceptual

```
In [15]: y_harm, y_perc = librosa.effects.hpss(audio_file)
```

```
plt.figure(figsize = (16, 6))  
plt.plot(y_harm, color = '#A300F9');  
plt.plot(y_perc, color = '#FFB100');
```



- Tempo BPM

```
In [17]: tempo, _ = librosa.beat.beat_track(y=y, sr = sr)  
tempo
```

```
Out[17]: array([107.66601562])
```

- Spectral Centroid

```
In [19]: # Calculate the Spectral Centroids  
spectral_centroids = librosa.feature.spectral_centroid(y=audio_file, sr=sr)[0]  
  
# Shape is a vector  
print('Centroids:', spectral_centroids, '\n')  
print('Shape of Spectral Centroids:', spectral_centroids.shape, '\n')  
  
# Computing the time variable for visualization  
frames = range(len(spectral_centroids))  
  
# Converts frame counts to time (seconds)  
t = librosa.frames_to_time(frames)  
  
print('frames:', frames, '\n')  
print('t:', t)  
  
# Function that normalizes the Sound Data
```

```
def normalize(x, axis=0):
    return sklearn.preprocessing.minmax_scale(x, axis=axis)
```

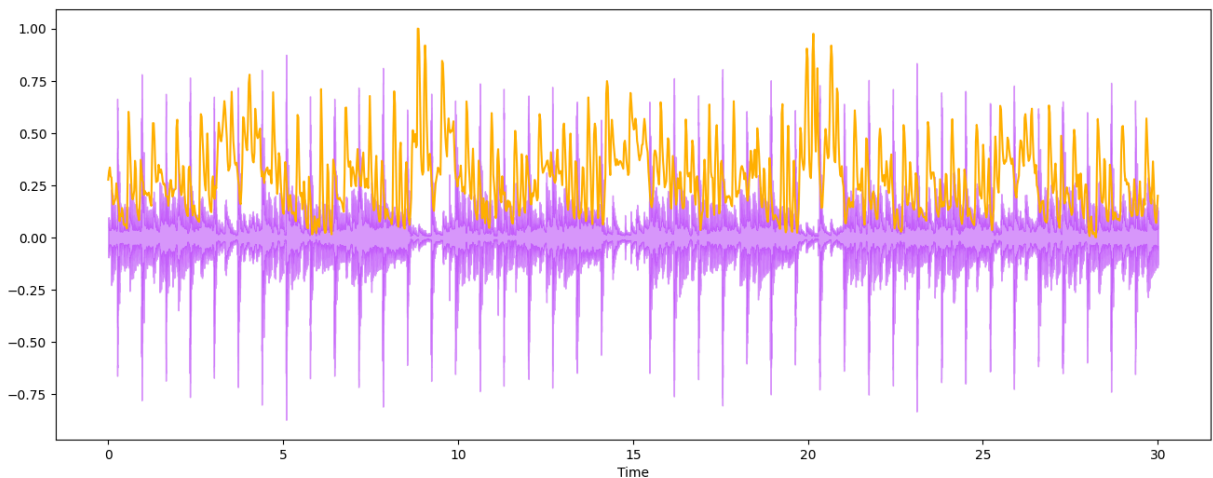
```
Centroids: [1758.29476821 1946.74243805 2038.81133429 ... 766.50415702 1041.0772875
3
1391.05146262]
```

Shape of Spectral Centroids: (1293,)

frames: range(0, 1293)

```
t: [0.00000000e+00 2.32199546e-02 4.64399093e-02 ... 2.99537415e+01
2.99769615e+01 3.00001814e+01]
```

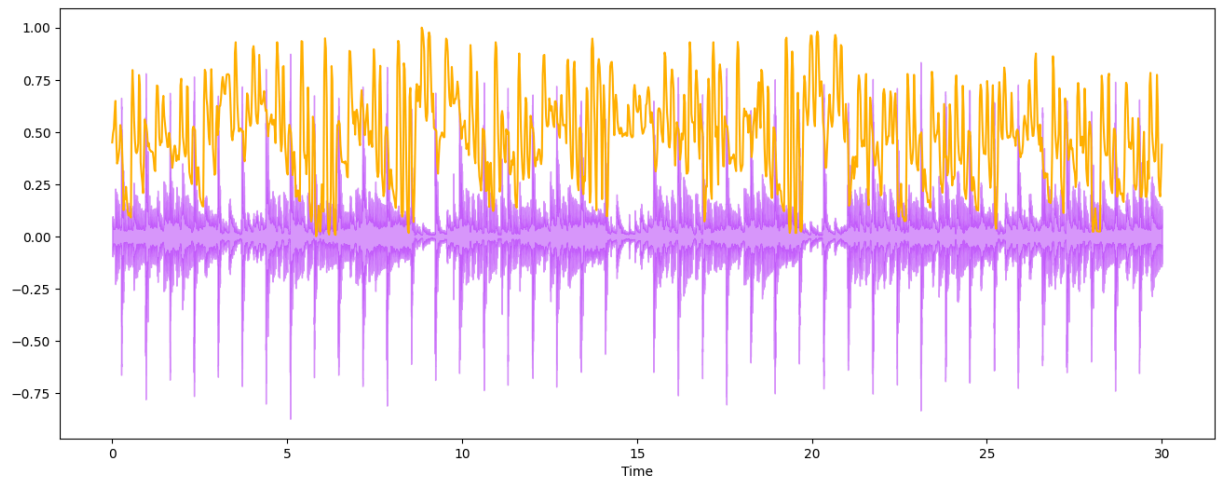
```
In [21]: #Plotting the Spectral Centroid along the waveform
plt.figure(figsize = (16, 6))
librosa.display.waveshow(audio_file, sr=sr, alpha=0.4, color = '#A300F9');
plt.plot(t, normalize(spectral_centroids), color='#FFB100');
```



- Spectral Rolloff

```
In [24]: # Spectral Rolloff Vector
spectral_rolloff = librosa.feature.spectral_rolloff(y=audio_file, sr=sr)[0]

# The plot
plt.figure(figsize = (16, 6))
librosa.display.waveshow(audio_file, sr=sr, alpha=0.4, color = '#A300F9');
plt.plot(t, normalize(spectral_rolloff), color='#FFB100');
```

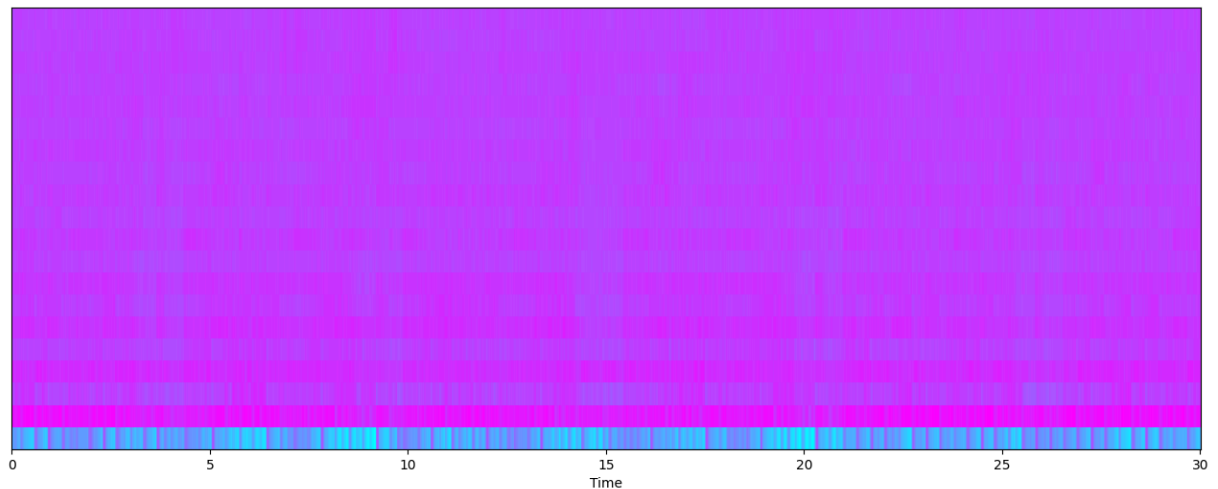


- Mel Frequency Cepstral Coefficients

```
In [26]: mfccs = librosa.feature.mfcc(y=audio_file, sr=sr)
print('mfccs shape:', mfccs.shape)

#Displaying the MFCCs:
plt.figure(figsize = (16, 6))
librosa.display.specshow(mfccs, sr=sr, x_axis='time', cmap = 'cool');
```

mfccs shape: (20, 1293)

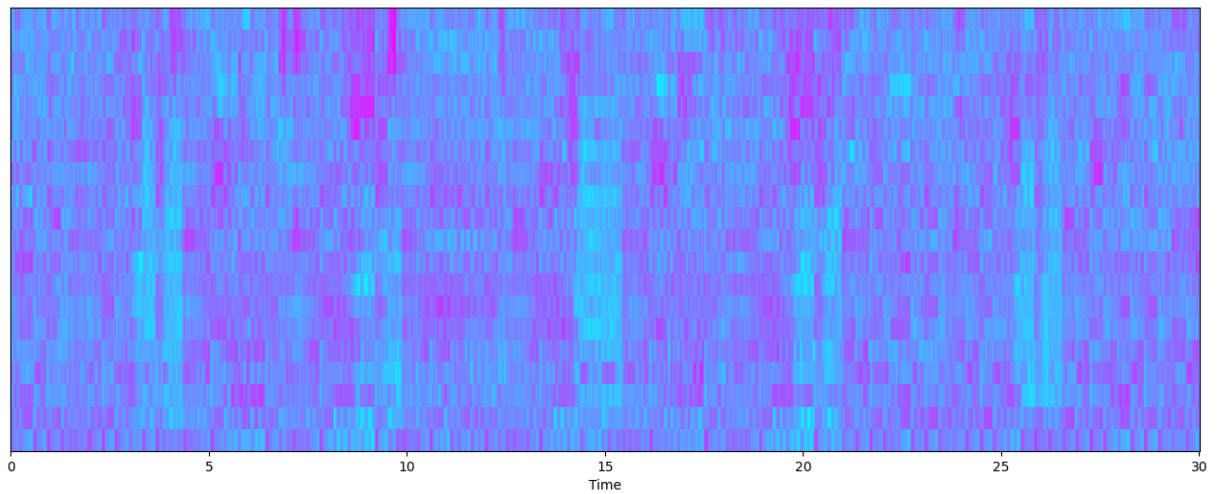


```
In [27]: # Perform Feature Scaling
mfccs = sklearn.preprocessing.scale(mfccs, axis=1)
print('Mean:', mfccs.mean(), '\n')
print('Var:', mfccs.var())

plt.figure(figsize = (16, 6))
librosa.display.specshow(mfccs, sr=sr, x_axis='time', cmap = 'cool');
```

Mean: -1.032594e-09

Var: 1.0000001



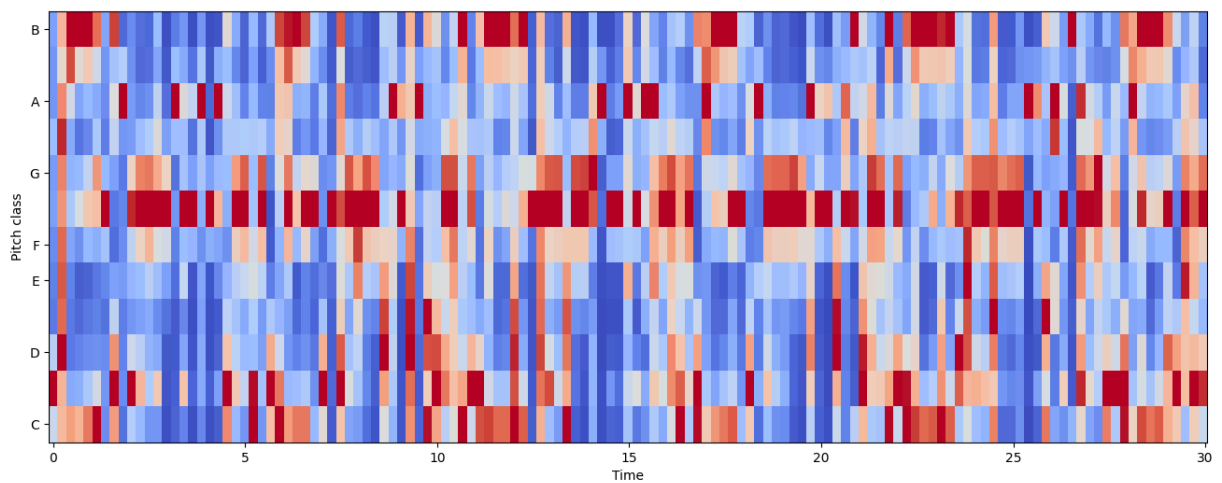
- Chroma frequencies

```
In [29]: # Increase or decrease hop_length to change how granular you want your data to be
hop_length = 5000

# Chromogram
chromagram = librosa.feature.chroma_stft(y=audio_file, sr=sr, hop_length=hop_length)
print('Chromogram shape:', chromagram.shape)

plt.figure(figsize=(16, 6))
librosa.display.specshow(chromagram, x_axis='time', y_axis='chroma', hop_length=hop
```

Chromogram shape: (12, 133)



EDA (Exploratory Data Analysis)

```
In [30]: data = pd.read_csv(f'{general_path}/features_30_sec.csv')
data.head()
```


Out[30]:

	filename	length	chroma_stft_mean	chroma_stft_var	rms_mean	rms_var	spectr
0	blues.00000.wav	661794	0.350088	0.088757	0.130228	0.002827	
1	blues.00001.wav	661794	0.340914	0.094980	0.095948	0.002373	
2	blues.00002.wav	661794	0.363637	0.085275	0.175570	0.002746	
3	blues.00003.wav	661794	0.404785	0.093999	0.141093	0.006346	
4	blues.00004.wav	661794	0.308526	0.087841	0.091529	0.002303	

5 rows × 60 columns

```
In [31]: # Computing the Correlation Matrix
spike_cols = [col for col in data.columns if 'mean' in col]
corr = data[spike_cols].corr()

# Generate a mask for the upper triangle
mask = np.triu(np.ones_like(corr, dtype=np.bool))

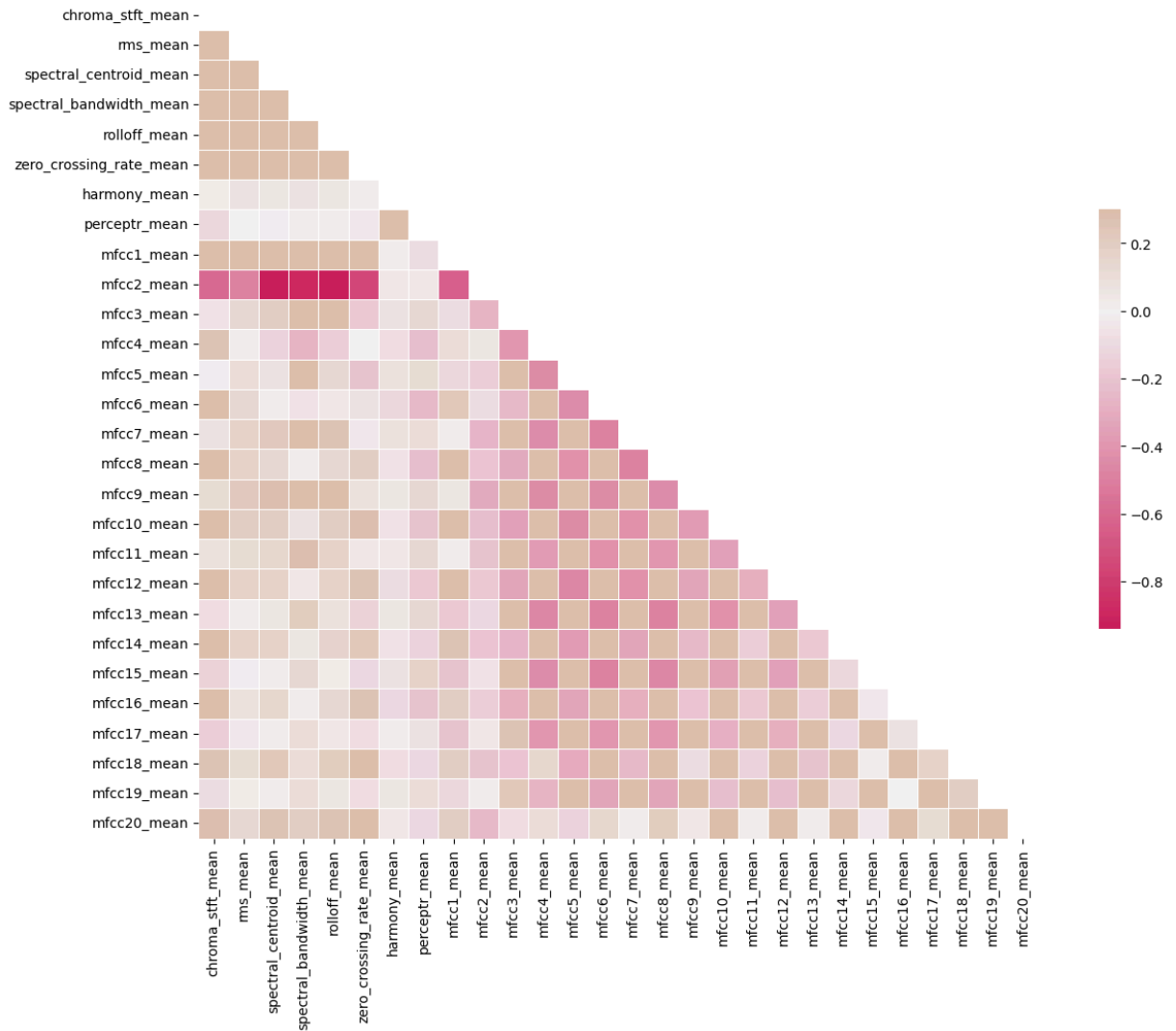
# Set up the matplotlib figure
f, ax = plt.subplots(figsize=(16, 11));

# Generate a custom diverging colormap
cmap = sns.diverging_palette(0, 25, as_cmap=True, s = 90, l = 45, n = 5)

# Draw the heatmap with the mask and correct aspect ratio
sns.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0,
            square=True, linewidths=.5, cbar_kws={"shrink": .5})

plt.title('Correlation Heatmap (for the MEAN variables)', fontsize = 25)
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10);
plt.savefig("Corr Heatmap.jpg")
```

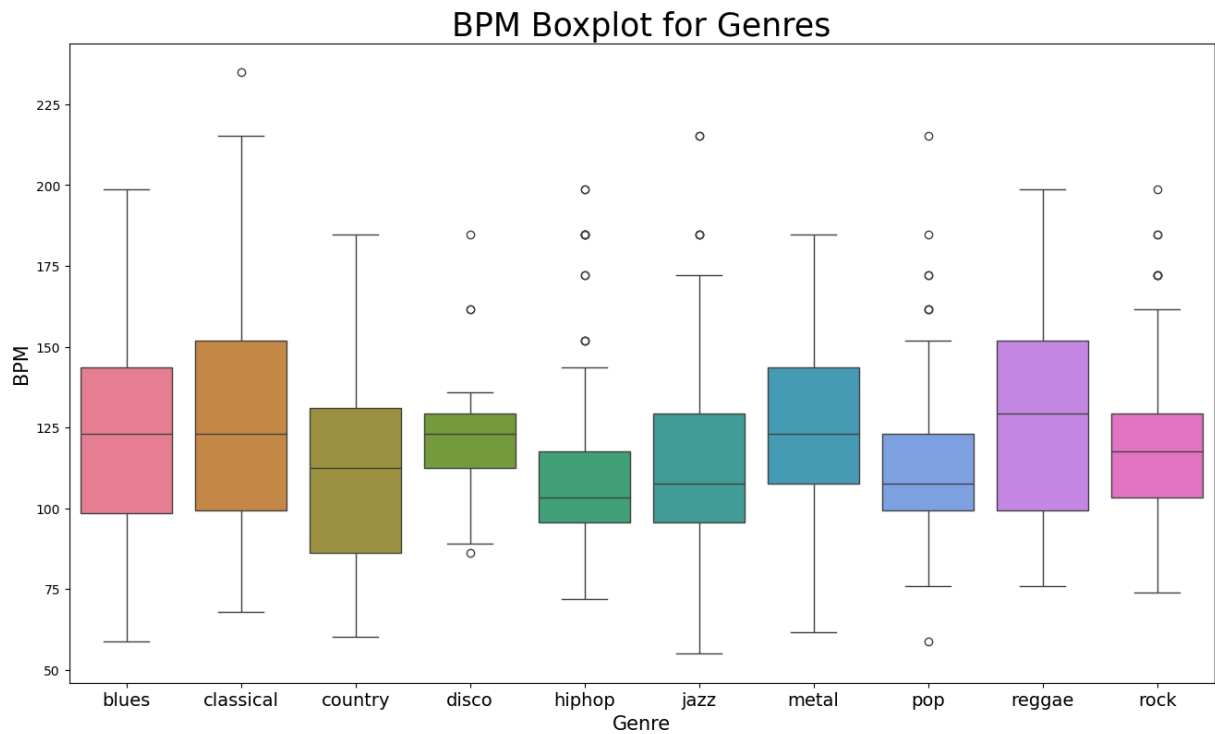
Correlation Heatmap (for the MEAN variables)



```
In [32]: x = data[["label", "tempo"]]

f, ax = plt.subplots(figsize=(16, 9));
sns.boxplot(x = "label", y = "tempo", data = x, palette = 'husl');

plt.title('BPM Boxplot for Genres', fontsize = 25)
plt.xticks(fontsize = 14)
plt.yticks(fontsize = 10);
plt.xlabel("Genre", fontsize = 15)
plt.ylabel("BPM", fontsize = 15)
plt.savefig("BPM Boxplot.jpg")
```



```
In [33]: from sklearn import preprocessing

data = data.iloc[0:, 1:]
y = data['label']
X = data.loc[:, data.columns != 'label']

#### NORMALIZE X ####
cols = X.columns
min_max_scaler = preprocessing.MinMaxScaler()
np_scaled = min_max_scaler.fit_transform(X)
X = pd.DataFrame(np_scaled, columns = cols)

#### PCA 2 COMPONENTS ####
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
principalComponents = pca.fit_transform(X)
principalDf = pd.DataFrame(data = principalComponents, columns = ['principal compon

# concatenate with target label
finalDf = pd.concat([principalDf, y], axis = 1)

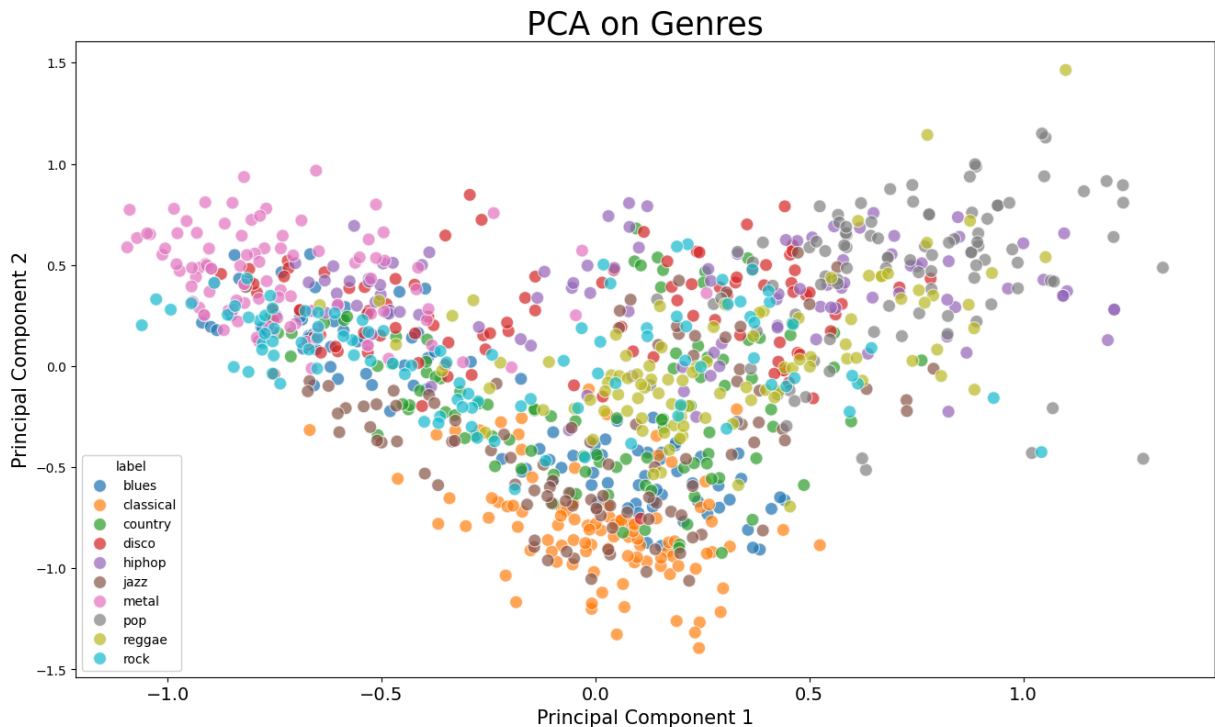
pca.explained_variance_ratio_

# 44.93 variance explained
```

```
Out[33]: array([0.2439355 , 0.21781804])
```

```
In [34]: plt.figure(figsize = (16, 9))
sns.scatterplot(x = "principal component 1", y = "principal component 2", data = fi
               s = 100);
```

```
plt.title('PCA on Genres', fontsize = 25)
plt.xticks(fontsize = 14)
plt.yticks(fontsize = 10);
plt.xlabel("Principal Component 1", fontsize = 15)
plt.ylabel("Principal Component 2", fontsize = 15)
plt.savefig("PCA Scattert.jpg")
```



Machine Learning Classification

```
In [35]: from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import SGDClassifier, LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier, XGBRFClassifier
from xgboost import plot_tree, plot_importance

from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score, roc_curve
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import RFE

In [36]: data = pd.read_csv(f'{general_path}/features_3_sec.csv')
data = data.iloc[0:, 1:]
data.head()
```

Out[36]:

	length	chroma_stft_mean	chroma_stft_var	rms_mean	rms_var	spectral_centroid_mean
0	66149	0.335406	0.091048	0.130405	0.003521	1773.065032
1	66149	0.343065	0.086147	0.112699	0.001450	1816.693777
2	66149	0.346815	0.092243	0.132003	0.004620	1788.539719
3	66149	0.363639	0.086856	0.132565	0.002448	1655.289045
4	66149	0.335579	0.088129	0.143289	0.001701	1630.656199

5 rows × 59 columns



- Features and Target variable

```
In [37]: y = data['label'] # genre variable.
X = data.loc[:, data.columns != 'label'] #select all columns but not the labels

#### NORMALIZE X ####

# Normalize so everything is on the same scale.

cols = X.columns
min_max_scaler = preprocessing.MinMaxScaler()
np_scaled = min_max_scaler.fit_transform(X)

# new data frame with the new scaled data.
X = pd.DataFrame(np_scaled, columns = cols)
```

- Splitting the data into training and testing

```
In [38]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

- Trying 8 different models to assess their performance

```
In [39]: def model_assess(model, title = "Default"):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    #print(confusion_matrix(y_test, preds))
    print('Accuracy', title, ': ', round(accuracy_score(y_test, preds), 5), '\n')
```

```
In [41]: # Naive Bayes
nb = GaussianNB()
model_assess(nb, "Naive Bayes")

# Stochastic Gradient Descent
sgd = SGDClassifier(max_iter=5000, random_state=0)
model_assess(sgd, "Stochastic Gradient Descent")
```

```

# KNN
knn = KNeighborsClassifier(n_neighbors=19)
model_assess(knn, "KNN")

# Decission trees
tree = DecisionTreeClassifier()
model_assess(tree, "Decission trees")

# Random Forest
rforest = RandomForestClassifier(n_estimators=1000, max_depth=10, random_state=0)
model_assess(rforest, "Random Forest")

# Support Vector Machine
svm = SVC(decision_function_shape="ovo")
model_assess(svm, "Support Vector Machine")

# Logistic Regression
lg = LogisticRegression(random_state=0, solver='lbfgs', multi_class='multinomial')
model_assess(lg, "Logistic Regression")

# Neural Nets
nn = MLPClassifier(solver='lbfgs', alpha=1e-5, hidden_layer_sizes=(5000, 10), random_state=0)
model_assess(nn, "Neural Nets")

```

Accuracy Naive Bayes : 0.51952

Accuracy Stochastic Gradient Descent : 0.65532

Accuracy KNN : 0.80581

Accuracy Decission trees : 0.64097

Accuracy Random Forest : 0.81281

Accuracy Support Vector Machine : 0.75409

Accuracy Logistic Regression : 0.6977

Accuracy Neural Nets : 0.67701

```

In [42]: # Final model
rforest = RandomForestClassifier(n_estimators=1000, max_depth=10, random_state=0)
rforest.fit(X_train, y_train)

preds = rforest.predict(X_test)

print('Accuracy', ':', round(accuracy_score(y_test, preds), 5), '\n')

# Confusion Matrix
confusion_matr = confusion_matrix(y_test, preds) #normalize = 'true'
plt.figure(figsize = (16, 9))
sns.heatmap(confusion_matr, cmap="Blues", annot=True,
            xticklabels = ["blues", "classical", "country", "disco", "hiphop", "jazz"])

```

```
yticklabels=["blues", "classical", "country", "disco", "hiphop", "jazz",
plt.savefig("conf matrix")
```

Accuracy : 0.81281



Documentation

Summary

In this project we are classifying songs by using 3 seconds long audio features from them.

For this project we used 8 different models in order to determine which one is best suited for this task.

Implementation

First thing was to explore audio data an features in order to determine what can be used for classification.

For the audio data we have done 2D representations of sound waves, fourier transforms, spectrograms and mel spectrograms.

The features we looked on were zero crossing rate, harmonics and perceptual, tempo bpm, spectral centroid, spectral rolloff, mel frequency cepstral coefficients (mfcc) and chroma frequencies.

Then comes the exploratory data analysis (EDA). We looked at the correlation heatmap for feature means, box plot for genres distribution and principal component analysis (to visualize possible groups of genres using normalization, PCA and scatter plots).

The next step is using the 8 different models to get the results. We are trying to predict the genre label from the dataset using a 70-30 split.

Results

The results are the following (in terms of accuracy):

Naive Bayes : 0.51952

Stochastic Gradient Descent : 0.65532

KNN : 0.80581

Decision trees : 0.64097

Random Forest : 0.81281

Support Vector Machine : 0.75409

Logistic Regression : 0.6977

Neural Nets : 0.67701

As shown above, the best performing one is Random Forest with 81% accuracy, closely followed by KNN with 80% and SVM with 75%.

The rest are all under 70%, with the worst performing one being Naive Bayes with only 51% accuracy.